SVM: Formulation, Interpretation & Analysis
SVMs for Linearly Non-Separable Data
SVM Kernel
Reinforcement Learning: Q-Learning & Temporal Difference Learning

Contents

1 SVM - Formulation, Interpretation & Analysis

1.1 Basic SVM Formulation (Linearly Separable Case)

Support Vector Machines find the optimal hyperplane that maximizes the margin between two classes. For linearly separable data, the goal is to maximize the margin $2/\|w\|$.

The primal optimization problem is:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \tag{1}$$

$$\text{subject to: } y_i(w^T x_i + b) \geq 1, \quad \forall i = 1, \ldots, n \tag{2}$$

where $x_i \in \mathbb{R}^d$ are training examples, $y_i \in \{-1, +1\}$ are labels, $w \in \mathbb{R}^d$ is the weight vector, and $b \in \mathbb{R}$ is the bias.

1.2 Dual Formulation & Derivation

1.3 Lagrangian with Complete KKT Constraints

Using Lagrange multipliers $\alpha_i \geq 0$, the Lagrangian is:

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i [y_i(w^T x_i + b) - 1] \tag{3}$$

Complete KKT Conditions:

1. **Stationarity**:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \tag{4}$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \tag{5}$$

2. **Primal feasibility**: $y_i(w^T x_i + b) \geq 1, \quad \forall i$

3. **Dual feasibility**: $\alpha_i \geq 0, \quad \forall i$

4. **Complementary slackness**:

$$\alpha_i [y_i(w^T x_i + b) - 1] = 0, \quad \forall i \in \{1, \ldots, n\} \tag{6}$$

Interpretation of (6)**:**

- If $\alpha_i = 0$: Point inside margin ($y_i(w^T x_i + b) > 1$)
- If $\alpha_i > 0$: Support vector on margin ($y_i(w^T x_i + b) = 1$)

Step 1: Differentiate w.r.t. w and b:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_{i=1}^{n} \alpha_i y_i x_i \tag{7}$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \quad \Rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \tag{8}$$

Step 2: Substitute back into Lagrangian (KKT conditions):

$$L(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^T x_j) \tag{9}$$

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \tag{10}$$

$$\text{s.t. } \alpha_i \geq 0, \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \tag{11}$$

where $K(x_i, x_j) = x_i^T x_j$ is the kernel function.

1.4 Decision Function & Support Vectors

The decision function is:

$$f(x) = \text{sign}\left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \right) \tag{12}$$

where SV denotes support vectors ($\alpha_i > 0$).

1.5 Geometric Interpretation

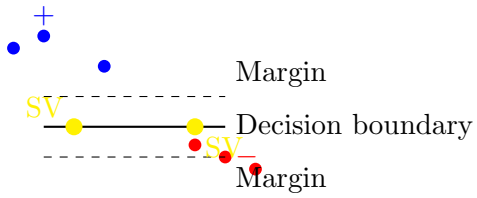

Figure 1: SVM Geometry: Margin $= 2/\|w\|$, Support vectors lie on margin boundaries

2 Dataset and Problem Setup

Consider the linearly separable dataset:

$$\mathbf{x}_1 = (1, 1), y_1 = +1 \qquad \mathbf{x}_3 = (-1, -1), y_3 = -1$$
$$\mathbf{x}_2 = (2, 3), y_2 = +1 \qquad \mathbf{x}_4 = (-2, -1), y_4 = -1$$

3 Gram Matrix Computation

The kernel matrix K is defined by $K_{ij} = y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$:

$$K = \begin{bmatrix} 2 & 5 & 2 & 3 \\ 5 & 13 & 5 & 7 \\ 2 & 5 & 2 & 3 \\ 3 & 7 & 3 & 5 \end{bmatrix}$$

4 KKT Analytical Solution via Matrix Form

To find the optimal α, we assume $\mathbf{x}_2$ and $\mathbf{x}_4$ are the Support Vectors (SVs) and solve the stationarity condition $\nabla_\alpha L = 0$. In matrix form:

$$\begin{bmatrix}1\\1\end{bmatrix} - \begin{bmatrix}K_{22} & K_{24}\\ K_{42} & K_{44}\end{bmatrix}\begin{bmatrix}\alpha_2\\ \alpha_4\end{bmatrix} - \lambda\begin{bmatrix}y_2\\ y_4\end{bmatrix} = \mathbf{0}$$

Given $y_2 = 1, y_4 = -1$ and the equality constraint $\sum \alpha_i y_i = 0 \implies \alpha_2 = \alpha_4 = a$:

$$1 - (13a + 7a) - \lambda = 0 \implies 1 - 20a = \lambda$$
$$1 - (7a + 5a) + \lambda = 0 \implies 1 - 12a = -\lambda$$

Adding the two equations to eliminate the Lagrange multiplier λ:

$$(1 - 20a) + (1 - 12a) = 0 \implies 2 - 32a = 0 \implies a = \frac{1}{16}$$

The resulting dual variables are:

$$\alpha = [0, \frac{1}{16}, 0, \frac{1}{16}]^T$$

5 Recovering Primal Parameters

Weight Vector w:

$$\mathbf{w} = \sum_{i=1}^{4} \alpha_i y_i \mathbf{x}_i = \frac{1}{16}(1)\begin{pmatrix}2\\3\end{pmatrix} + \frac{1}{16}(-1)\begin{pmatrix}-2\\-1\end{pmatrix} = \begin{pmatrix}1/4\\1/4\end{pmatrix}$$

Bias b: Using SV $\mathbf{x}_2$: $y_2(\mathbf{w} \cdot \mathbf{x}_2 + b) = 1$

$$1\left(\frac{1}{4}(2) + \frac{1}{4}(3) + b\right) = 1 \implies \frac{5}{4} + b = 1 \implies b = -\frac{1}{4}$$

6 Verification

The decision function is $f(\mathbf{x}) = \frac{1}{4}x_1 + \frac{1}{4}x_2 - \frac{1}{4}$.

- For $\mathbf{x}_1$: $y_1 f(\mathbf{x}_1) = 1(\frac{1}{4} + \frac{1}{4} - \frac{1}{4}) = 0.25 < 1$.

Observation: Since x_1 violates the margin, the true SV set must include $\mathbf{x}_1$. A matrix system 3×3 that includes $\alpha_1, \alpha_2, \alpha_4$ would be the next step for a perfectly valid margin.

7 SVMs for Linearly Non-Separable Data (Soft Margin SVM)

For non-separable data, introduce slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \tag{13}$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \tag{14}$$

Interpretation of C:

- $C \to \infty$: Hard margin (no violations allowed)
- $C \to 0$: Ignore misclassifications (maximize margin regardless)

7.1 Soft Margin Dual

The dual becomes identical to hard margin but with box constraints $0 \leq \alpha_i \leq C$:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \tag{15}$$

$$\text{s.t. } 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \tag{16}$$

KKT Conditions reveal three types of points:

- $\alpha_i = 0$: Points outside margin (not SV)
- $0 < \alpha_i < C$: Support vectors on margin
- $\alpha_i = C$: Misclassified points or boundary violations

8 SVM Kernel Methods

Kernel trick: Replace dot products with kernel function $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$.

9 SVM Kernel Methods: Polynomial Kernel

The **Polynomial Kernel** is a non-linear kernel used when the relationship between features is not linear. It is particularly effective in image processing and scenarios where feature interactions (like $x_1 x_2$) are significant.

9.1 Mathematical Definition

The kernel function is defined as:

$$K(x_i, x_j) = (\gamma x_i^T x_j + r)^d, \quad \gamma > 0 \tag{17}$$

where d is the polynomial degree, r is a free parameter, and γ is a scale parameter.

9.2 Example: Circular (Non-Linear) Separation

Consider a "donut" dataset where one class is surrounded by another. Using a quadratic kernel ($d = 2$), the SVM can create a circular decision boundary.

9.3 Comparison with Linear Kernel

Unlike the linear kernel ($K = x_i^T x_j$), the polynomial kernel considers the **interactions** between features up to degree d. For $d = 2$ and two features (x_1, x_2), the mapping $\phi(x)$ effectively looks like:

$$\phi(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2] \tag{18}$$

This allows the model to fit curves and complex shapes that a single line cannot capture.

9.4 Kernel Matrix (Gram Matrix)

For n training points:

$$K_{ij} = K(x_i, x_j), \quad K \in \mathbb{R}^{n \times n} \tag{19}$$

Mercer Condition: K must be positive semi-definite.

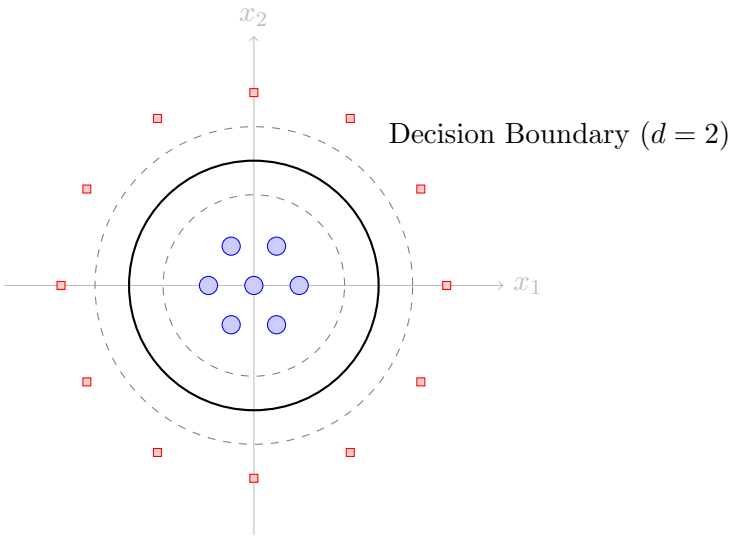

Figure 2: A polynomial kernel ($d = 2$) generating a non-linear circular boundary in the original 2D space.

Kernel	**Function**
Linear	$K(x_i, x_j) = x_i^T x_j$
Polynomial	$K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$
RBF/Gaussian	$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
Sigmoid	$K(x_i, x_j) = \tanh(\gamma x_i^T x_j + r)$

Table 1: Common SVM Kernels

10 Reinforcement Learning

Reinforcement Learning (RL) is a paradigm of machine learning in which an **agent** learns to make decisions by interacting with an **environment**. At each time step, the agent observes the current **state**, performs an **action**, and receives a **reward**. The objective of the agent is to learn a policy that maximizes the expected cumulative reward over time.

11 The Markov Decision Process (MDP)

Reinforcement Learning (RL) is a framework where an **Agent** learns to make decisions by interacting with an **Environment**. The goal is to learn a policy that maximizes the cumulative **Reward**. The mathematical foundation of RL is the MDP, defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: State space (all possible situations).
- $\mathcal{A}$: Action space (all possible moves).
- $P(s'|s, a)$: Transition probability (likelihood of reaching s' from s via action a).
- $R(s, a, s')$: Reward function (feedback received for a transition).
- $\gamma \in [0, 1)$: Discount factor (weighting of future vs. immediate rewards).

11.1 Value Functions

The **Action-value function** $q_\pi(s, a)$ represents the expected return starting from state s, taking action a, and following policy π thereafter:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \;\middle|\; S_t = s, A_t = a \right] \tag{20}$$

The **Bellman Expectation Equation** decomposes this recursively:

$$q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \;\mid\; S_t = s, A_t = a] \tag{21}$$

12 Q-Learning (Off-Policy TD Control)

Q-Learning is a model-free algorithm that directly approximates the optimal action-value function q_* by updating the Q-table based on the **Temporal Difference (TD) Error** δ_t.

12.1 The Update Rule

The Q-value for a state-action pair is updated after every transition:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{22}$$

where α is the learning rate and $\gamma \max_a Q(S_{t+1}, a)$ represents the discounted value of the best possible future action.

12.2 Q-Learning Algorithm

Algorithm 1 Q-Learning (Off-Policy TD Control)

Initialize $Q(s,a)$ arbitrarily (e.g., $Q(s,a) = 0, \forall s \in \mathcal{S}, a \in \mathcal{A}$)
Initialize hyperparameters: learning rate $\alpha \in (0,1]$, discount factor $\gamma \in [0,1]$
for each episode **do**
  Initialize state S
  repeat
    Choose action A from S using policy derived from Q (e.g., ϵ-greedy)
    Take action A, observe reward R and next state S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S,A) \right]$
    $S \leftarrow S'$
  until S is terminal
end for

13 Practical Example: Grid World

Consider a 4×4 grid where an agent seeks the shortest path to a goal while avoiding walls.

13.1 Environment Configuration

- **Start State:** S_{13} — **Goal State:** S_4 (Reward: $+100$)
- **Pit/Wall:** S_8 (Reward: -100) — **Standard Step:** Reward: -1
- **Parameters:** $\alpha = 0.5$, $\gamma = 0.9$

Table 2: Grid World State Layout

Row	Col 1	Col 2	Col 3	Col 4
A	S_1	S_2	S_3	S_4 **(Goal)**
B	S_5	S_6	S_7	S_8 **(Wall)**
C	S_9	S_{10}	S_{11}	S_{12}
D	S_{13} **(Start)**	S_{14}	S_{15}	S_{16}

13.2 Step-by-Step Numerical Update

Assume the agent is at S_3 and moves **East** into the Goal (S_4):

1. Current state $S = S_3$, Action $A =$ East, Reward $R = 100$.
2. Since S_4 is terminal, $\max_a Q(S_4, a) = 0$.
3. Update: $Q(S_3, \text{East}) \leftarrow 0 + 0.5[100 + 0.9(0) - 0] = \mathbf{50.0}$.

In a later episode, the agent moves from S_2 to S_3:

1. $R = -1$ (step penalty). $\max_a Q(S_3, a)$ is now 50.0.
2. Update: $Q(S_2, \text{East}) \leftarrow 0 + 0.5[-1 + 0.9(50.0) - 0] = \mathbf{22.0}$.

14 Convergence Guarantees

Q-Learning converges to the optimal action-value function q_* with probability 1, provided that the **Robbins-Monro** conditions are satisfied:

1. $\sum_{t=1}^{\infty} \alpha_t = \infty$ (The step size is large enough to reach any state).
2. $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$ (The step size eventually reduces to filter out noise).